

Lab 8: A Pipeline from Commit to Running

Build the whole chain: a push runs tests, builds an image, and deploys it, with no human step in the middle.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

DURATION

One 2-hour block

MARKS

16

SUBMIT

icclab8-<ROLLNO>.pdf

OFFERING

Odd Semester 2026-27

The target

```
git push
├─ lint          seconds
├─ tests         a minute
├─ build image   minutes
├─ push to registry
├─ deploy to staging
└─ smoke test
```

Use GitHub Actions, GitLab CI or any equivalent. The concepts are the same.

Part 1: Continuous integration (5 marks)

1.1 A workflow that runs on every push: lint, then tests. Show a green run.

1.2 Push a deliberate lint error. Show the run failing, and **how long it took to fail**.

1.3 Reorder so the slowest stage runs last if it does not already. Report the time-to-failure for a lint error before and after. Explain why fast checks belong first.

1.4 Make a test depend on something not pinned, for example an unpinned dependency version or the current date. Show it passing once and failing later, or explain precisely how it would. Then pin it. Explain why "it passes on the third rerun" is a failure, not a flake.

Part 2: Build once, promote (4 marks)

2.1 Build a container image in the pipeline and tag it with the **commit SHA**, not `latest`.

2.2 Push it to a registry.

2.3 Deploy **that exact tag** to staging. Show the deployed image digest matching the built one.

2.4 Explain, in three sentences, why rebuilding the image separately for staging and production means you did not test what you shipped.

Part 3: Secrets (3 marks)

3.1 Inject a registry credential and one application secret from the CI system's secret store. Show they do **not** appear in the logs.

3.2 Deliberately `echo` a secret in a step. Observe the masking. Then explain why masking is a safety net and not a control.

3.3 State three rules for handling secrets in a pipeline, and what each prevents.

Part 4: Gates and environments (2 marks)

4.1 Make deployment conditional: only from the default branch, only after tests pass.

4.2 Open a pull request with a failing test and show the deploy job does not run.

Part 5: Should merging build for real? (2 marks)

The digital-twin case study made this decision twice and got it wrong the first time.

5.1 State the argument for a plan-only pipeline, and its cost.

5.2 State the argument for applying for real on merge, asserting convergence, and destroying in the same run.

5.3 Explain why leaving the environment standing **only when the run fails** is the right exception, and implement that behaviour, or describe exactly how you would.

What to submit

`icclab8-<ROLLN0>.pdf` : your workflow file as text, screenshots or logs of a green run and a failed run, the time-to-failure comparison, the digest match from 2.3, the masked-secret log, and your Part 5 argument.

MARKS	FOR
5	Part 1: a working pipeline and the fast-feedback measurement
4	Part 2: build-once-promote demonstrated with a digest match
3	Part 3: secrets handled correctly and the masking limitation understood
2	Part 4: a real gate, demonstrated
2	Part 5: a reasoned position with the exception implemented or specified